

Architecture Decisions and Migration Paths — Cheat Sheet

Monolith vs. Microservices · Migration Paths · Layering · New-Project Recommended Stack

MIGRATION PATHS BY CURRENT ARCHITECTURE

Legacy-stack monolith → proxy + strangler fig; split presentation from backend first; anti-corruption layer; CDC for data overlap (Debezium).

Healthy monolith that outgrew itself → modularise first: modules by business capability, Spring Modulith, in-process domain events, per-module schema.

Modular monolith with a hot spot → extract that capability: new features as services first; vertical slice; ids not object references; replicate data back to the monolith; gateway routes to the new service; keep the monolith's own transaction pivot or retrieable; sequence the extraction order.

Partially extracted system → sagas / outbox / CQRS per Architectural Patterns; consolidate the gateway/BFF; retire the monolith, or keep it as one more service.

Single-tenant installs → control plane first → silo lift-and-shift → layered migration → service-by-service migration; choose silo/pool per service.

Each row names the primary technique for that starting point — most real systems pass through more than one of these rows on the way to their target architecture, in the order listed above.

Recommended extraction order once you are splitting a monolith into services, so the monolith's own remaining transaction stays a pivot or retrieable transaction at every step, never a compensatable one:

Orders → Customers → Catalogue/Inventory → Payments

New capabilities that do not fit the current domain model well are usually cheapest to build as a service from day one instead of extracting them later — Richardson's "implement new features as services" strategy.

MONOLITH VS. MICROSERVICES

Choose a monolith when...

- Small project, small team — one team around a table
- One-shot delivery or prototype
- App unlikely to evolve much
- Startup still searching for its business model
- Missing operational prerequisites — pipeline, monitoring

Choose microservices when...

- Large, complex, long-lived system
- Several autonomous teams need to release independently
- Components need very different scaling/resource profiles
- Fault isolation or independent release cadence is required
- Part of the system needs a different tech stack

Is it really the architecture? Slow, buggy releases are often a *process* problem — manual testing, no CI/CD — fix that first before blaming the architecture.

Signals to split: the monolithic-hell symptoms — intimidating complexity, a slow edit-build-test loop, a long commit-to-deploy path, conflicting scaling needs, no fault isolation, locked into an obsolete stack.

Signals to merge back: chatty inter-service calls and per-call infrastructure cost outweigh the split's benefit — the Prime Video audio/video monitoring case.

BENEFITS / INCONVENIENCES

Benefits (microservices)

- Independent teams & services
- Small, fast-to-evolve, independently deployable
- Independent scalability
- Fault isolation
- Freedom to adopt new tech

Inconveniences

- Loss of ACID tx & consistent queries
- Distributed-system complexity
- Far more code & infra to maintain
- Cross-service feature coordination
- Right boundaries are hard to find

LAYERING

SPA + native apps → gateway/BFF → API → services (+ DB). One API (REST, or GraphQL/gRPC) behind a gateway; independent web, native and partner clients all share it.

Server-rendered UI is still fine for: internal tools, small teams, content sites, one-shot projects — Thymeleaf, Vaadin, Blazor Server, JSF.

What it costs once you split:

- CORS between the API and the separate frontend origin
- Two deployables to build, version and release
- API versioning discipline, ideally contract-first (OpenAPI)
- Authentication moves to a BFF, not the browser itself

PITFALLS

Big bang rewrite

— "the only thing it guarantees is a big bang," never a working delivery.

Distributed monolith

— services that still have to deploy together, with none of the monolith's simplicity.

Shared database

— couples services through their data model instead of decoupling them.

Premature Kubernetes

— orchestration overhead before there is enough to orchestrate.

Tech-first SaaS

— multi-tenancy retrofitted after the fact costs far more than designing it in.

POINTERS

Pattern catalogues

microservices.io/patterns · martinowler.com/bliki · martinowler.com/articles

Cloud guidance

Azure Architecture Center · AWS Prescriptive Guidance & SaaS Lens · Google Cloud Architecture Center

Books

Richardson, *Microservices Patterns* (Manning) · Golding, *Building Multi-Tenant SaaS Architectures* (O'Reilly, free AWS whitepaper edition)

NEW PROJECT → RECOMMENDED STACK

SCENARIO	ARCHITECTURE	BACKEND	API	MESSAGING	DATABASE	FRONTEND	MOBILE	DEPLOYMENT
(a) Small one-shot / internal tool	Plain monolith	Spring Boot	— (UI-bound)	None	One relational DB	Thymeleaf or small React/Vue	—	Single JAR / container / PaaS
(b) Startup MVP, must evolve fast	Modular monolith	Spring Boot	REST	None yet	PostgreSQL	React / Angular / Vue	Native, added later	Container, managed platform
(c) Long-lived line-of-business app	Modular monolith	Spring Boot + Modulith, or Quarkus	REST	RabbitMQ (back-ground work)	Relational DB	Server-rendered UI	—	Container
(d) High-traffic consumer app	Gateway/BFF + microservices on hot paths	Spring Boot (extracted services)	REST via gateway	Kafka	Elasticsearch/Solr + cache + relational	React / Angular / Vue	Native, via gateway	Kubernetes (hot paths)
(e) Large multi-team enterprise platform	Microservices by business capability	Spring Boot per service	API-first REST/gRPC + BFF per client	Event-driven (domain events)	DB per service	React / Angular / Vue per BFF	Native + partner APIs	Kubernetes, distributed tracing
(f) Multi-tenant SaaS product	Control plane + pooled application plane	Spring Boot	REST (control + app plane)	As needed per service	Tiered, partitioned data	React / Angular / Vue	Native, tenant-aware	Serverless control plane; containers for app plane
(g) Batch / data-processing job	No web layer	Spring Batch	—	—	Relational or file-based source/target	—	—	Scheduler-invoked process